

Intelligent Vehicle Telemetry & AI Diagnostics: An IoT-Based OBD-II Ecosystem

Muhammad Shaheer Rashad (22L-6587)
Ramisha Kamal Pasha (22L-6804)
Muhammad Aneeb ur Rehman (22L-6808)

Department of Computer Science
FAST-NUCES

May 6, 2026

Contents

1	Project Justification	3
1.1	Abstract	3
1.2	Introduction & Problem Statement	3
1.3	Project Rigor & Feasibility	4
1.3.1	Project Complexity and Rigor	4
1.3.2	Justification for IoT Integration	4
1.3.3	Feasibility Analysis	5
2	Objectives & Goals	6
2.1	Primary Objectives	6
2.2	Success Criteria	6
2.3	Goals	6
3	Requirements & Components	7
3.1	Hardware Components	7
3.2	Software Stack	7
4	Methodology	9
4.1	High-Level Implementation Plan	9
4.2	Data Flow (Block Diagram)	10
4.3	Proposed Data Processing Approach	10
5	Software Requirements Specification (SRS)	12
5.1	Purpose and Scope	12
5.2	Functional Requirements	12
5.3	Non-Functional Requirements	13
6	Design Document	15
6.1	Purpose	15
6.2	High-Level Architecture Diagram	15
6.3	Hardware Design	17
6.3.1	Software Design	17
6.3.2	Database Schema	20
7	Description of Data Processing and Diagnostic Logic	22
7.0.1	Problem Formulation	22
7.0.2	Approach Selection and Justification	22
7.0.3	Features Used	22
7.0.4	Implementation Details	23

7.0.5	Output Metrics	23
7.0.6	Discussion	23
8	Description of ML/DL Algorithms Used	24
8.1	Problem Formulation	24
8.2	Algorithm Selection & Justification	25
8.2.1	Selected Algorithm: LSTM Autoencoder	25
8.2.2	Why Not Simpler Approaches?	25
8.3	Model Architecture	26
8.3.1	Encoder (Compression)	26
8.3.2	Decoder (Reconstruction)	26
8.3.3	Input Features	26
8.4	Implementation Details	27
8.4.1	Libraries and Frameworks	27
8.4.2	Training Dataset	28
8.4.3	Data Preprocessing Pipeline	28
8.4.4	Data Split	28
8.4.5	Training Configuration	29
8.4.6	Anomaly Threshold Derivation	29
8.4.7	Scoring and Risk Classification	30
8.4.8	Supplementary: Rule-Based Explanations	30
8.5	Performance Metrics	30
8.5.1	Evaluation Approach	30
8.5.2	Metrics on the Synthetic Test Set	31
8.5.3	Real-World Deployment Results	31
8.5.4	Interpreting the Results	32
8.5.5	Challenges and Limitations	32
8.5.6	What This Means Practically	32

Chapter 1

Project Justification

1.1 Abstract

In the modern automotive landscape, vehicles function as complex "computers on wheels," yet a significant gap exists between the vast amounts of data generated by Electronic Control Units (ECUs) and the practical utility of this information for the average owner. Current On-Board Diagnostics (OBD-II) systems are primarily reactive, triggering alerts only after a component has already failed. This project addresses these challenges by developing an intelligent, IoT-based OBD-II telemetry ecosystem designed for real-time monitoring and predictive maintenance.

The proposed solution utilizes an OBD-II sensor with built-in Bluetooth connectivity to extract live metrics such as Engine RPM, Coolant Temperature, and Intake MAP from the vehicle's CAN bus. A Raspberry Pi receives the telemetry stream over Bluetooth, stores it in CSV format, and forwards the session data to a FastAPI backend.

To bridge the interpretation gap, the system organizes raw telemetry into structured trip records and backend-generated summaries that are easier to interpret than raw CSV logs. The backend performs data validation, session analysis, and threshold-based diagnostic checks on key signals. The expected outcome is a seamless, end-to-end platform that helps users review vehicle health trends, reduces manual inspection effort, and makes vehicle diagnostics more accessible to non-technical users.

1.2 Introduction & Problem Statement

In the modern automotive landscape, vehicles have evolved from purely mechanical machines into complex "computers on wheels," generating vast amounts of data through hundreds of Electronic Control Units (ECUs). However, a significant gap exists between the availability of this data and its practical utility for the average vehicle owner.

Currently, On-Board Diagnostics (OBD-II) systems are primarily reactive. They rely on Diagnostic Trouble Codes (DTCs), which trigger a Check Engine Light (CEL) only after a component has already failed or exceeded a critical threshold. Reactive maintenance is significantly more costly than predictive strategies, as unplanned downtime and secondary damage can substantially increase overall repair costs.

Furthermore, while DIY OBD-II loggers are available, they typically produce thousands of rows of raw CSV data that require professional automotive expertise to interpret. This creates a barrier for everyday users who lack the technical knowledge to extract

meaningful insights from such data.

The problem can be summarized into two key challenges:

1. **The Interpretation Gap:** Raw telemetry data (e.g., RPM, fuel trims, MAP sensor readings) is often unintelligible to most users, leading to “data fatigue,” where critical early warning signs are overlooked or ignored.
2. **The Latency Gap:** By the time a standard OBD-II code is generated, the underlying issue may have already caused significant damage. There is a need for a faster logging and reporting workflow that captures abnormal behavior early and makes it visible soon after the driving session ends.

This project addresses this gap by developing an OBD-II telemetry ecosystem centered on reliable data capture and backend analysis. The OBD-II sensor interfaces directly with the vehicle’s CAN bus to extract live metrics such as Engine RPM, Coolant Temperature, and MAP. These readings are transmitted over Bluetooth to a Raspberry Pi, which logs the session to CSV and uploads it to the backend server. The backend then validates the data, computes session statistics, and generates readable diagnostic reports, allowing users to proactively monitor vehicle health.

1.3 Project Rigor & Feasibility

1.3.1 Project Complexity and Rigor

- **Hardware and Protocol Interfacing:** Establishing reliable communication with a vehicle’s CAN bus (ISO 15765-4) through a Bluetooth-enabled OBD-II sensor requires correct PID polling, stable pairing, and loss-tolerant data capture over a 20-minute logging session.
- **Backend Data Processing:** Moving beyond simple threshold-based alerts (e.g., “if Temp > 100”), the project requires building a robust backend workflow. The backend must parse large CSV datasets, compute useful statistics, and apply rule-based checks across RPM, Pressure, and Temperature to identify suspicious operating conditions.
- **Full-Stack Integration:** Building a dependable workflow from the Bluetooth OBD-II sensor to the Raspberry Pi and then to the FastAPI server requires robust device pairing, local buffering, networking, and API design.

1.3.2 Justification for IoT Integration

An IoT approach is mandatory because collecting and organizing thousands of telemetry samples requires a connected OBD-to-Pi-to-server architecture. The Raspberry Pi handles Bluetooth data collection and temporary storage, while the FastAPI backend handles parsing, validation, and report generation. This division of responsibilities keeps the in-vehicle setup lightweight while still supporting detailed post-drive analysis.

The combined Raspberry Pi and backend workflow also improves reliability. The Raspberry Pi can buffer telemetry locally during the trip, and the backend can later

transform the uploaded CSV data into readable summaries, statistics, and flagged observations. This approach is simpler to implement, easier to maintain, and better aligned with the project timeline.

1.3.3 Feasibility Analysis

The project is highly feasible within the given timeframe:

- **Defined Hardware Scope:** By restricting acquisition to a few critical metrics and delegating storage and upload duties to the Raspberry Pi, the project remains technically manageable within the academic timeline.
- **Hardware Availability:** Bluetooth-enabled OBD-II scanners and Raspberry Pi boards are low-cost, readily available components.
- **Existing Architecture:** The team can concentrate efforts on telemetry extraction, Raspberry Pi data forwarding, server-side ingestion, and backend report generation.

Furthermore, the team possesses relevant skills in programming (C++, Python) and backend development, which align well with the project requirements. Existing libraries for Bluetooth communication, CSV processing, and backend API development further reduce development overhead, ensuring that the system can be implemented, tested, and refined within the academic timeframe.

Chapter 2

Objectives & Goals

2.1 Primary Objectives

- **Data Acquisition:** Successfully interface with a vehicle's CAN bus through a Bluetooth-enabled OBD-II sensor to extract key engine metrics at steady intervals.
- **Session Logging & Forwarding:** Implement a reliable system for the Raspberry Pi to receive telemetry over Bluetooth, log driving sessions into CSV format, and forward the files to the backend server.
- **Backend Diagnostics:** Develop a backend workflow that ingests the CSV data, performs validation and rule-based analysis, and generates easily understandable diagnostic summaries for the end-user.

2.2 Success Criteria

The project will be considered successful if:

1. The OBD-II sensor and Raspberry Pi gateway can continuously capture and log data for a minimum of 20 minutes without connection loss or data corruption.
2. The Raspberry Pi successfully forwards the recorded CSV data to the backend server after a driving session.
3. The backend correctly processes the dataset and returns a valid JSON response containing session statistics, flagged conditions, and a human-readable diagnostic summary.

2.3 Goals

Define what success looks like (e.g., successful CAN bus communication, accurate diagnostic reports, low-latency Edge inference).

Chapter 3

Requirements & Components

3.1 Hardware Components

The hardware foundation of this project centers around a Bluetooth-enabled OBD-II sensor and a Raspberry Pi gateway for local collection and forwarding of vehicle telemetry. All physical components have been selected for their affordability, availability, and suitability for lightweight automotive data acquisition.

- **Bluetooth OBD-II Sensor:** The primary in-vehicle interface that connects to the car's OBD-II port and exposes telemetry from the CAN bus over built-in Bluetooth.
- **Raspberry Pi 3:** The local device responsible for pairing with the OBD-II sensor, collecting telemetry, storing it as CSV, and transmitting the session data to the FastAPI backend.
- **Power Supply and Storage:** The Raspberry Pi uses its onboard storage and external power source to support stable logging during the driving session.
- **OBD-II Extension Cable (16-pin):** A male-to-female diagnostic cable spliced to extract CAN signals (Pins 6 and 14) and Chassis/Signal Ground (Pins 4/5) without permanently modifying the vehicle's internal wiring harness.

3.2 Software Stack

The software stack spans across three distinct layers: Sensor Interface, Raspberry Pi Gateway, and Backend Processing.

- **Bluetooth OBD-II Interface Software:** The sensor-side communication stack provides access to standard OBD-II PIDs over Bluetooth and serves as the source of telemetry for the gateway.
- **Python on Raspberry Pi:** Python scripts on the Raspberry Pi manage Bluetooth pairing, PID polling, CSV logging, retry logic, and secure forwarding of session data to the backend server.
- **Python & FastAPI Framework:** A high-performance Python web framework responsible for receiving the CSV datasets from the Raspberry Pi via a REST API endpoint. It manages data ingestion and pre-processing.

- **Pandas Library (Python):** A critical data manipulation library used on the backend to parse the telemetry CSV, calculate driving session averages, and condense the dataset before sending it to the ML layer.
- **PostgreSQL Database:** The database layer used to store uploaded telemetry metadata, processed session records, and generated diagnostic summaries.

Chapter 4

Methodology

4.1 High-Level Implementation Plan

1. **Hardware setup and validation:** Connect the Bluetooth OBD-II sensor and Raspberry Pi; validate stable PID polling and pairing on the target vehicle protocol.
2. **Telemetry acquisition:** Poll RPM, Coolant Temperature, and MAP at fixed intervals (4Hz) and decode frames into usable values.
3. **Edge logging:** Store timestamped telemetry to CSV on the Raspberry Pi during the session.
4. **Wireless synchronization:** Receive telemetry over Bluetooth on the Raspberry Pi and queue it for backend upload.
5. **Backend ingestion:** Upload CSV from the Raspberry Pi to the FastAPI backend, parse with Pandas, and compute summary features.
6. **Backend diagnostics:** Run rule-based checks and session analysis to generate user-friendly diagnostic insights.
7. **Result delivery:** Store diagnostics in the database and expose reports through the server layer.

4.2 Data Flow (Block Diagram)

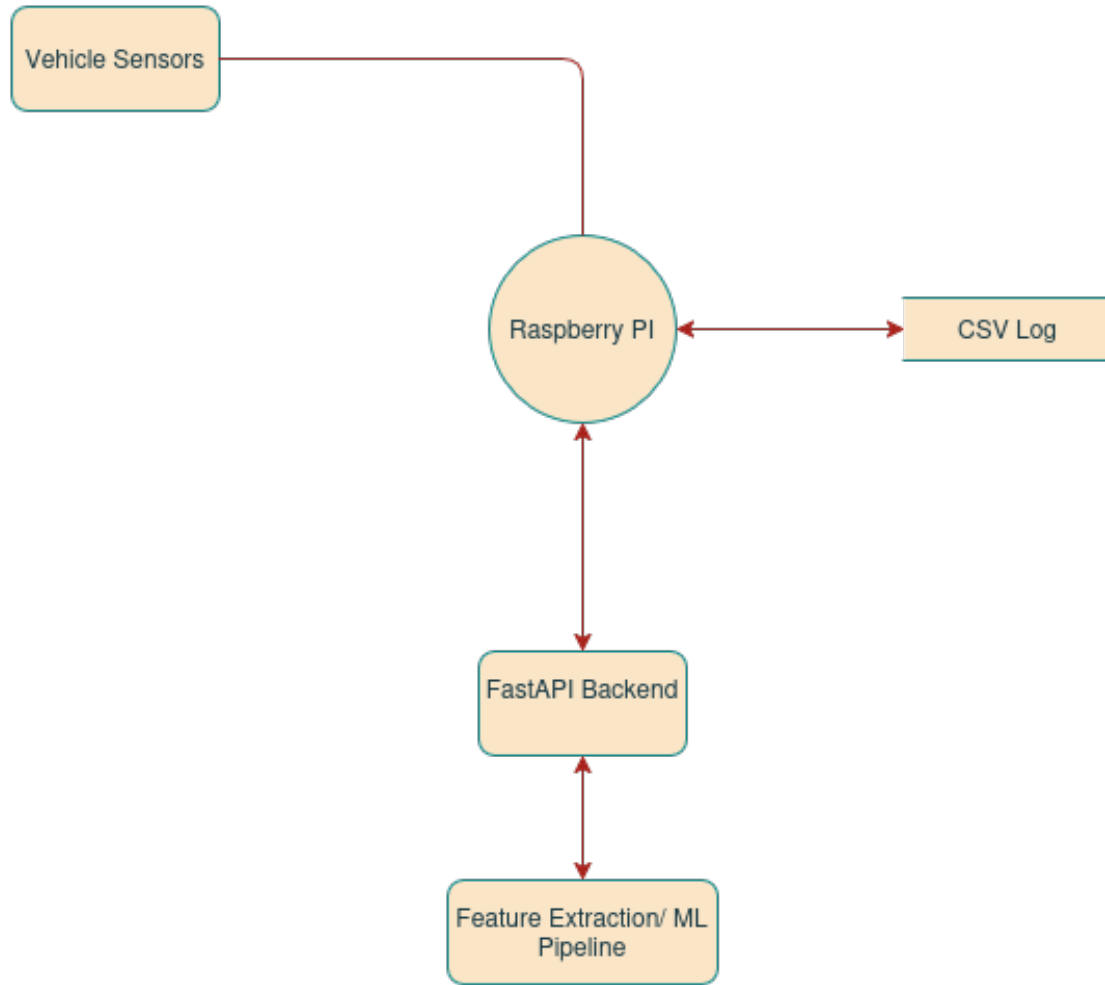


Figure 4.1: Data flow diagram of the IoT-based OBD-II telemetry ecosystem.

4.3 Proposed Data Processing Approach

- **Input features:** Use RPM, Coolant Temperature, and MAP along with timestamps and basic derived statistics such as averages, maxima, and rates of change.
- **Validation logic:** Check for missing values, malformed rows, duplicated records, and inconsistent sampling intervals before processing.
- **Diagnostic logic:** Apply threshold-based checks and simple cross-signal comparisons to identify overheating, unstable idle behavior, or unusual pressure readings.
- **Session summary:** Compute trip-level statistics and flagged events so each uploaded session produces a concise and readable report.
- **Why suitable:** This approach is easier to implement, easier to explain, and sufficient for an academic prototype focused on reliable telemetry capture and reporting.

- **User-facing output:** Return session statistics, flagged signals, and a short backend-generated diagnostic summary.

Chapter 5

Software Requirements Specification (SRS)

5.1 Purpose and Scope

Purpose: This document defines the required behavior and quality attributes of the Intelligent Vehicle Telemetry & Diagnostics system, covering OBD-II data collection, Raspberry Pi gateway forwarding, and backend reporting.

In scope:

- Real-time acquisition of vehicle metrics (PIDs) from OBD-II/CAN.
- Telemetry capture from the OBD-II sensor and local CSV session logging on the Raspberry Pi.
- Wireless Bluetooth transfer from the OBD-II sensor to the Raspberry Pi, followed by upload to the backend.
- Backend-side session analysis and user-readable diagnostic summary generation.

Out of scope:

- Direct vehicle control (e.g., braking, throttle, steering, ECU reprogramming).
- Guaranteed detection of every mechanical fault in all vehicle models.
- Full dealership-grade diagnostics for all manufacturer-specific PIDs.

5.2 Functional Requirements

- FR-1** The system shall initialize CAN communication (ISO 15765-4) and maintain continuous telemetry capture during an active session.
- FR-2** The Raspberry Pi gateway shall sample or receive RPM, Coolant Temperature, and MAP at a target rate of 4Hz and timestamp each sample.
- FR-3** The Raspberry Pi gateway shall store telemetry locally as CSV and preserve data if backend upload is temporarily unavailable.

- FR-4** The Raspberry Pi gateway shall receive telemetry from the OBD-II sensor over Bluetooth during an active session.
- FR-5** The Raspberry Pi gateway shall upload completed session logs to a FastAPI endpoint.
- FR-6** The backend shall associate uploaded CSV logs with the correct trip session and validate the received metadata.
- FR-7** The backend shall parse uploaded CSV data, validate required fields, and reject malformed files with clear error messages.
- FR-8** The backend shall compute telemetry statistics and identify threshold violations or flagged conditions for each session.
- FR-9** The backend shall generate a concise diagnostic summary and return/store results in structured JSON format.
- FR-10** The system shall make diagnostic status, anomaly highlights, and summary text available through the server response layer.

5.3 Non-Functional Requirements

Performance

1. The Raspberry Pi gateway shall sample or receive RPM, Coolant Temperature, and MAP at a stable 4Hz target rate.
2. Time drift between consecutive samples should remain low enough to preserve time-series analysis quality.
3. End-to-end processing (CSV upload to diagnostic response) should complete within an acceptable user wait time under normal network conditions.

Reliability

1. The gateway shall support at least 20 minutes of continuous logging without crash or Bluetooth disconnection.
2. If upload fails, telemetry shall remain safely stored on the Raspberry Pi for deferred transfer.
3. The backend shall validate CSV structure and reject corrupted or incomplete files with clear error messages.

Security

1. User access to diagnostics shall require authenticated sessions (JWT-based validation).
2. Data transmission between the Raspberry Pi and FastAPI backend shall use HTTPS/TLS encryption.

3. Sensitive secrets (API keys, database credentials, Bluetooth pairing credentials) shall not be hard-coded in deployed builds.

Chapter 6

Design Document

6.1 Purpose

This Design Document translates the SRS requirements into a practical system architecture that can be implemented across the OBD device, Raspberry Pi, and FastAPI backend.

6.2 High-Level Architecture Diagram

Component View

- **Hardware Layer:** OBD-II port and Bluetooth-enabled OBD-II sensor.
- **Gateway Layer:** Raspberry Pi receives telemetry over Bluetooth, stores trip logs, and uploads them securely.
- **Backend Layer:** FastAPI service, telemetry parsing logic, reporting engine, and PostgreSQL storage.
- **Presentation Layer:** Server responses expose anomaly status, summary text, and maintenance recommendations.

Protocol View

- **OBD-II/CAN (ISO 15765-4):** Vehicle ECU to Bluetooth OBD-II sensor telemetry polling.
- **Bluetooth Serial / ELM327 Protocol:** OBD-II sensor to Raspberry Pi telemetry transfer.
- **HTTPS REST API:** Raspberry Pi to FastAPI backend for upload, processing, and retrieval.

Block Diagram (Textual)

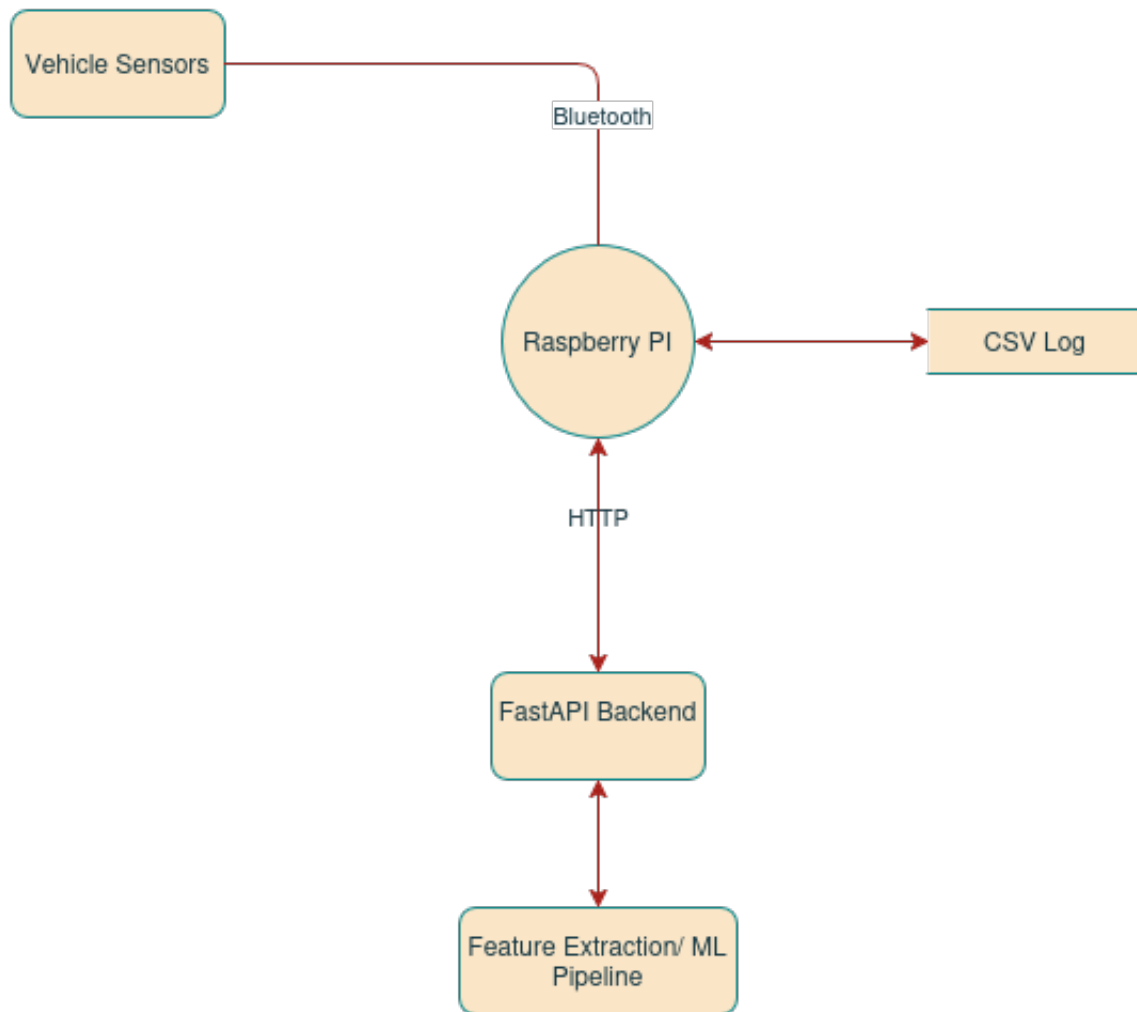


Figure 6.1: Block Diagram for connections & protocols used.

6.3 Hardware Design

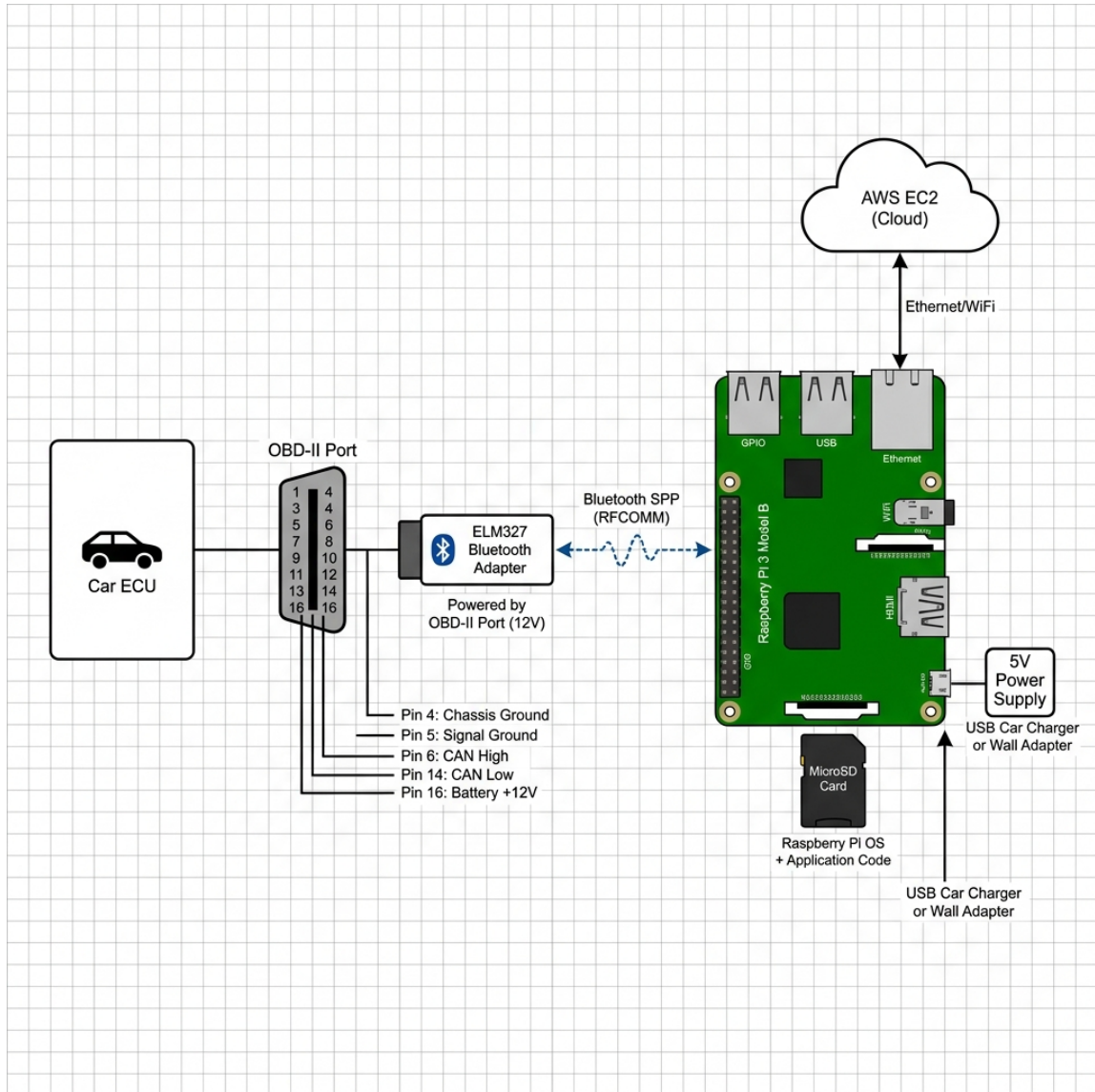


Figure 6.2: Circuit Diagram for Hardware components

6.3.1 Software Design

Data Flow Diagram (DFD)

The system follows a layered data flow architecture spanning three tiers: the vehicle (data source), the Raspberry Pi (edge processing), and the cloud (visualization). Figure 6.3 illustrates the complete data pipeline.

1. **Vehicle Layer:** The Engine Control Unit (ECU) exposes live sensor data through the standard 16-pin OBD-II diagnostic port. An ELM327 Bluetooth v2.1 adapter translates CAN bus frames into serial AT commands.
2. **Edge Layer (Raspberry Pi 3):** The `python-OBd` library reads from the Bluetooth RFCOMM serial device (`/dev/rfcomm0`). A PID Scheduler polls five PIDs at

1 Hz (Speed, RPM, Coolant Temperature, Throttle Position, Engine Load). Each reading is simultaneously:

- Persisted to a local SQLite database (`data/telemetry.db`) via SQLAlchemy ORM,
 - Written to daily CSV files (`data/YYYY-MM-DD.csv`) as a flat-file backup,
 - Broadcast to connected WebSocket clients in real time.
3. **API Layer:** A FastAPI server exposes RESTful endpoints and a WebSocket stream on port 8000, enabling both pull-based and push-based data access.
 4. **ML Layer:** An Isolation Forest model processes stored telemetry to generate per-session anomaly scores and diagnostic reports, persisted in the `diagnostic_reports` table.
 5. **Cloud Layer:** Grafana, deployed via Docker on AWS EC2, polls the FastAPI REST endpoints through the Infinity data source plugin and renders live dashboards with gauges, time-series graphs, and alert panels.

API Design

The FastAPI backend exposes the following endpoints:

- **GET /health**
 - *Purpose:* System health check — reports OBD-II connection status, reconnection count, polling errors, and uptime.
 - *Output:* JSON with `status`, `connected` (bool), `reconnect_count`, `poll_errors`, `uptime_seconds`.
- **GET /telemetry/latest**
 - *Purpose:* Retrieve the most recent value of each polled PID from the in-memory buffer.
 - *Output:* Array of `TelemetrySample` objects, each containing `timestamp`, `vehicle_session_id`, `pid`, `name`, `value`, and `unit`.
- **GET /telemetry/range?from_ts=&to_ts=&pid=**
 - *Purpose:* Query historical telemetry data from the SQLite database with optional time range and PID filters.
 - *Input:* Query parameters — `from_ts` (ISO 8601), `to_ts` (ISO 8601), `pid` (e.g., `SPEED`).
 - *Output:* Filtered array of `TelemetrySample` objects.
- **GET /telemetry/plot?pid=SPEED**
 - *Purpose:* Server-rendered time-series visualization using Chart.js — useful for quick inspection without Grafana.

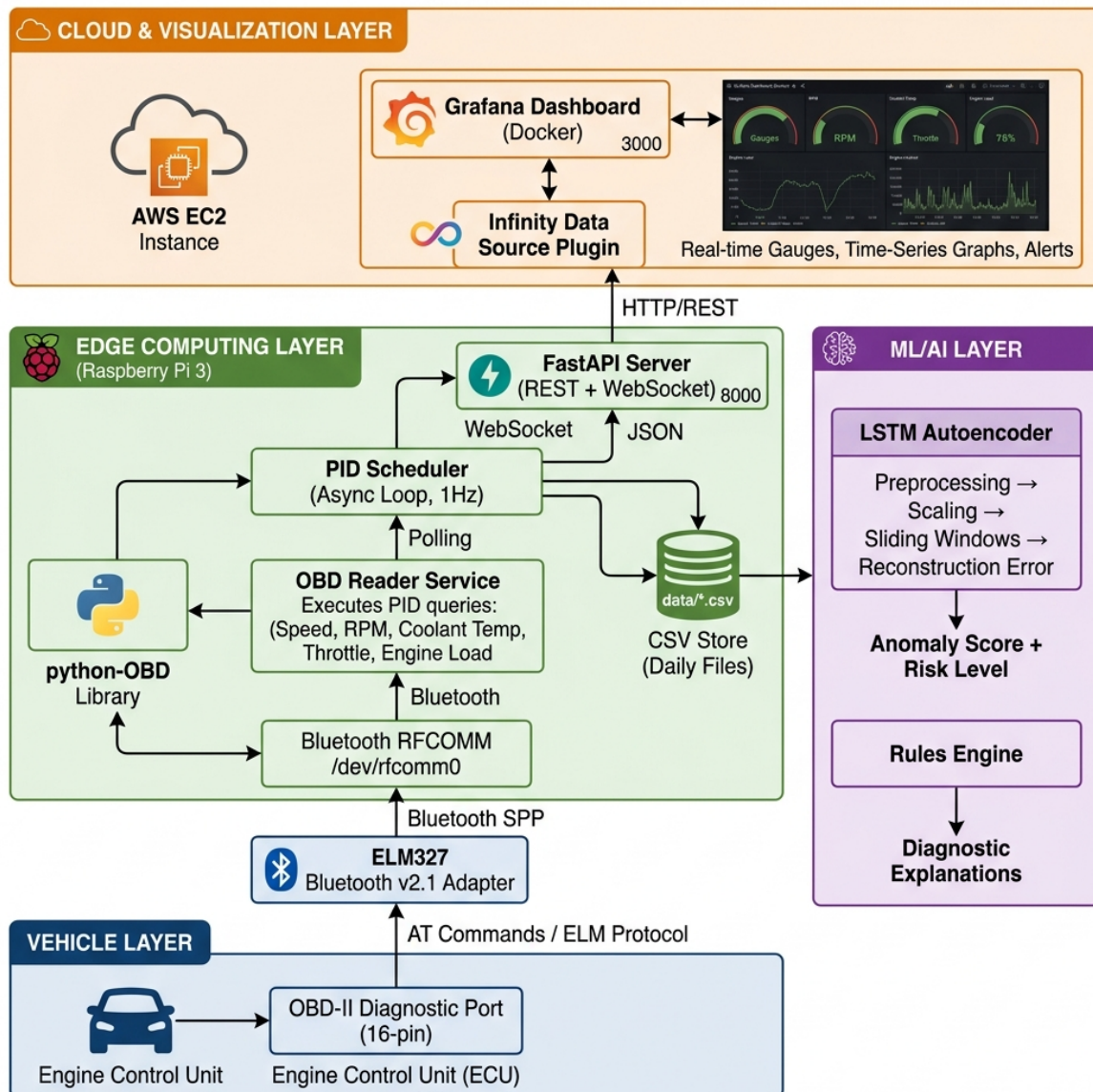


Figure 6.3: Data flow diagram of the IoT-based OBD-II telemetry ecosystem.

- *Output*: HTML page with an interactive line chart.
- **WebSocket /telemetry/stream**
 - *Purpose*: Real-time push-based data stream. Each new PID reading is broadcast as a JSON message to all connected clients.
 - *Protocol*: WebSocket (RFC 6455). Messages are serialized via Pydantic’s `model_dump_json()`.

All endpoints use Pydantic models for request validation and response serialization, with automatic OpenAPI (Swagger) documentation available at `/docs`.

6.3.2 Database Schema

The system uses SQLite as its persistence layer, chosen for its zero-configuration deployment on the Raspberry Pi (no separate database server process). The ORM layer is implemented using SQLAlchemy 2.0. The database file is stored at `data/telemetry.db`. The schema consists of three tables with the following structure:

- **vehicle_sessions**: Tracks individual driving sessions, each identified by a UUID generated when the system starts or reconnects to the OBD-II adapter.
- **telemetry_readings**: Stores individual PID samples with timestamps, linked to their parent session via a foreign key. Indexed on `session_id`, `timestamp`, and `pid` for efficient range queries.
- **diagnostic_reports**: Stores the output of the Isolation Forest anomaly detection model, including the computed anomaly score, severity classification, a human-readable summary, and the model version used.

Table 6.1: Database schema — `vehicle_sessions` table.

Column	Type	Description
<code>id</code>	INTEGER (PK)	Auto-incrementing primary key
<code>session_uuid</code>	VARCHAR(36)	Unique session identifier (UUID v4)
<code>start_time</code>	DATETIME	Timestamp of first reading in session
<code>end_time</code>	DATETIME	Timestamp of last reading in session
<code>reading_count</code>	INTEGER	Total PID readings in this session
<code>created_at</code>	DATETIME	Record creation timestamp

Figure ?? shows the entity-relationship diagram for the database schema. The `telemetry_readings` and `diagnostic_reports` tables both reference `vehicle_sessions` via foreign keys, establishing a one-to-many relationship: each session contains multiple readings and may have one or more diagnostic reports generated by different model versions.

Data Volume

Table 6.4 summarizes the database contents after importing telemetry data from the initial deployment.

Table 6.2: Database schema — `telemetry_readings` table.

Column	Type	Description
<code>id</code>	INTEGER (PK)	Auto-incrementing primary key
<code>session_id</code>	INTEGER (FK)	References <code>vehicle_sessions.id</code>
<code>timestamp</code>	DATETIME	When the reading was captured
<code>pid</code>	VARCHAR(32)	OBD-II PID code (e.g., <code>SPEED</code> , <code>RPM</code>)
<code>name</code>	VARCHAR(64)	Human-readable name (e.g., <code>engine_rpm</code>)
<code>value</code>	FLOAT	Numeric sensor value
<code>unit</code>	VARCHAR(16)	Unit of measurement (e.g., <code>km/h</code> , <code>rpm</code> , <code>°C</code>)

Table 6.3: Database schema — `diagnostic_reports` table.

Column	Type	Description
<code>id</code>	INTEGER (PK)	Auto-incrementing primary key
<code>session_id</code>	INTEGER (FK)	References <code>vehicle_sessions.id</code>
<code>anomaly_score</code>	FLOAT	Mean Isolation Forest decision score
<code>severity</code>	VARCHAR(16)	Risk classification: <code>LOW</code> / <code>MEDIUM</code> / <code>HIGH</code>
<code>summary</code>	TEXT	Human-readable diagnostic summary
<code>model_used</code>	VARCHAR(64)	ML model identifier and version
<code>created_at</code>	DATETIME	When the analysis was performed

Table 6.4: Database statistics after initial data import.

Metric	Value
Total driving sessions	11
Total telemetry readings	77,435
Diagnostic reports generated	11
Database file size	10.6 MB
PIDs per reading cycle	5
Severity breakdown: <code>LOW</code>	5 sessions
Severity breakdown: <code>MEDIUM</code>	5 sessions
Severity breakdown: <code>HIGH</code>	1 session

Chapter 7

Description of Data Processing and Diagnostic Logic

7.0.1 Problem Formulation

The problem is formulated as a **telemetry processing and diagnostic reporting task** for identifying unusual engine behavior, particularly signs of overheating or inefficient performance.

Instead of relying on machine learning models, the system focuses on collecting reliable OBD-II telemetry, validating uploaded records, and applying deterministic checks to detect suspicious operating conditions. The objective is to transform raw sensor logs into readable summaries that help users understand vehicle behavior over a driving session.

7.0.2 Approach Selection and Justification

Approach Used: Rule-based diagnostic analysis with session statistics

This approach was selected because it is transparent, easy to implement, and well suited to an academic prototype built around structured OBD-II telemetry.

Justification:

- **Interpretability:** Each flagged condition can be traced back to an explicit threshold, trend, or consistency check.
- **Low Complexity:** The backend can process uploaded sessions without requiring model training, external AI services, or specialized inference infrastructure.
- **Practicality:** RPM, temperature, and pressure readings can still provide useful insight when summarized with well-defined engineering rules.

7.0.3 Features Used

The diagnostic logic uses the following OBD-II sensor features:

- Engine Coolant Temperature
- Engine RPM
- Intake Manifold Absolute Pressure (MAP)

- Timestamp of each reading

These features are sufficient for computing session averages, peaks, variability, and simple cross-signal checks.

7.0.4 Implementation Details

Libraries and Frameworks:

- Python and FastAPI for backend development
- Pandas and NumPy for data handling and summary computation

Data Processing Pipeline:

1. Receive CSV data from the Raspberry Pi gateway
2. Validate schema, timestamps, and missing values
3. Compute summary statistics such as average RPM, peak temperature, and signal ranges
4. Apply threshold-based checks to identify flagged conditions
5. Generate a concise session report in JSON format

7.0.5 Output Metrics

The backend report includes processing-oriented metrics such as:

- Total samples received
- Session duration
- Average and peak values for key signals
- Count of flagged observations

7.0.6 Discussion

This approach prioritizes reliability, simplicity, and interpretability over predictive intelligence. While it does not learn hidden patterns the way a machine learning model might, it is easier to validate and better aligned with the current project scope.

Key Challenges:

- Sensor noise and occasional missing values
- Vehicle-to-vehicle variation in normal operating ranges
- Choosing thresholds that are informative without being overly sensitive

Future Improvements:

- Refine threshold values using locally collected driving sessions
- Expand the feature set with additional OBD-II parameters
- Add richer visualization and historical comparison across trips

Chapter 8

Description of ML/DL Algorithms Used

8.1 Problem Formulation

The core challenge we set out to tackle with this project was: *how do you detect that something is going wrong with a vehicle’s engine, before the driver even notices?*

Specifically, we wanted to predict engine overheating events from OBD-II sensor data. On the surface, this sounds like a straightforward classification problem — just label some data as “overheating” and some as “normal,” train a classifier, and you’re done. But in practice, it’s not that simple at all.

The fundamental issue is that real-world vehicle failure data is almost impossible to collect ethically and safely. Nobody intentionally drives their car until the engine overheats just to generate training labels. And even if they did, every vehicle, driver, and driving environment behaves differently — so those labels wouldn’t generalize well. In technical terms, we’re dealing with a **severe class imbalance problem**: normal driving data is abundant, but genuine anomaly data is virtually nonexistent.

This is what led us to frame the problem as **unsupervised anomaly detection** rather than supervised classification. Instead of trying to teach the model *what overheating looks like*, we teach it *what normal driving looks like*. Then, anything the model has never seen before — any pattern that deviates significantly from what it considers “normal” — gets flagged as potentially anomalous.

Concretely, the model learns to **reconstruct** sequences of 7 OBD-II sensor readings from normal driving. When presented with new data, it tries to reconstruct that data the same way. If the **reconstruction error** (the gap between the original input and what the model produces) is unusually high, it suggests that the input doesn’t match learned normal patterns — which could indicate overheating, abnormal engine load, or some other fault developing.

This is a well-established approach in predictive maintenance for industrial systems, and it maps naturally onto our vehicle telemetry problem where we have plenty of “healthy” data but essentially zero labeled failure events.

8.2 Algorithm Selection & Justification

8.2.1 Selected Algorithm: LSTM Autoencoder

We used a **Long Short-Term Memory (LSTM) Autoencoder** — a deep learning architecture that combines two powerful ideas:

1. **LSTM Networks** excel at modeling sequential, time-dependent data. Engine behavior is inherently temporal: coolant temperature doesn't just spike instantaneously — it rises gradually over multiple readings, correlated with RPM, throttle position, and airflow patterns. A model needs to understand this temporal context to distinguish between “coolant is at 100°C because the car was driven normally for 30 minutes” versus “coolant jumped to 100°C in the last 2 minutes while the car was idling” — the first is expected; the second is concerning.
2. **Autoencoders** are neural networks trained to reconstruct their own input. The key insight is that they learn a compressed, internal representation of the data during this process. When trained on only normal data, they become very good at reconstructing normal patterns — and very bad at reconstructing anomalies they've never encountered. That failure to reconstruct (the high reconstruction error) becomes our anomaly signal.

8.2.2 Why Not Simpler Approaches?

We genuinely considered several alternatives before settling on the LSTM Autoencoder, and each had limitations that mattered for our specific use case:

Isolation Forest (a popular anomaly detection algorithm from scikit-learn) was actually our initial prototype. It's fast, lightweight, and works well for tabular data. However, it treats every data point independently — it has no concept of “this reading came right after that reading.” Engine overheating is a progressive phenomenon that unfolds over time, so an algorithm that can't model temporal relationships will miss the gradual degradation patterns that precede a failure. An Isolation Forest might catch an already-overheated engine, but it won't detect the *trend* leading up to it.

One-Class SVM shares this limitation of treating data points as independent observations. It also scales poorly — training time grows significantly with large datasets, which becomes a real problem when you're processing tens of thousands of OBD-II readings per driving session.

Simple threshold rules (e.g., “alert if coolant temperature exceeds 105°C”) are easy to implement and interpretable, but they can't capture the multi-dimensional correlations that matter in practice. A coolant temperature of 102°C might be perfectly normal after highway driving at 3500 RPM, but deeply alarming if the car has been idling at 800 RPM. Static thresholds either miss subtle anomalies or produce constant false alarms.

Statistical approaches (z-score, moving averages) assume the underlying data follows a predictable distribution — typically Gaussian. Vehicle telemetry data during real-world driving is anything but Gaussian. RPM fluctuates wildly in city traffic, throttle position is highly sporadic, and driving patterns change dramatically between highway cruising and stop-and-go traffic.

The LSTM Autoencoder handles all of these concerns: it models sequences (capturing temporal dynamics), processes multiple features simultaneously (capturing cross-feature

correlations), requires no labeled anomaly data (unsupervised), and makes no distributional assumptions about the data.

8.3 Model Architecture

The architecture follows a standard encoder-decoder pattern with LSTM layers as the backbone:

8.3.1 Encoder (Compression)

The encoder's job is to read a 50-timestep window of 7 OBD-II features and compress it into a dense, low-dimensional representation — essentially distilling the important patterns from 350 data points (50×7) into a vector of just 32 numbers.

```
Input Layer      -> (batch, 50, 7)      # 50 timesteps x 7 sensor features
LSTM (128 units) -> (batch, 50, 128)     # First recurrent layer, returns full sequence
LSTM (64 units)  -> (batch, 64)          # Second recurrent layer, returns final state
Dense (32 units) -> (batch, 32)          # Compressed "latent" representation
```

The first LSTM layer with 128 hidden units processes the full sequence and captures low-level temporal patterns (e.g., RPM oscillations, throttle fluctuations). The second LSTM with 64 units further compresses this, and the final Dense layer reduces it to a 32-dimensional latent vector. This bottleneck is critical — it forces the network to learn only the most essential patterns from the input, discarding noise.

8.3.2 Decoder (Reconstruction)

The decoder takes that compressed 32-dimensional vector and attempts to reconstruct the original 50-timestep input:

```
RepeatVector(50) -> (batch, 50, 32)    # Repeat latent vector across 50 timesteps
LSTM (64 units)  -> (batch, 50, 64)      # First decoder LSTM
LSTM (128 units) -> (batch, 50, 128)     # Second decoder LSTM (mirrors encoder)
Dense (7 units)  -> (batch, 50, 7)      # Reconstruct all 7 features per timestep
```

The decoder mirrors the encoder architecture in reverse. `RepeatVector(50)` broadcasts the latent vector to each of the 50 timesteps, and the LSTM layers progressively expand it back to the original dimensionality. The final `TimeDistributed(Dense(7))` layer produces a value for each of the 7 features at each timestep.

All LSTM layers use ReLU activation. The model is compiled with the **Adam optimizer** and **Mean Squared Error (MSE)** loss — the loss function directly measures how well the reconstruction matches the original input.

8.3.3 Input Features

The model operates on seven OBD-II parameters that together paint a comprehensive picture of engine thermal behavior:

These features were selected because overheating is rarely caused by a single sensor going out of range. It's usually a combination: coolant temperature climbing *while* RPM

#	Feature	Unit	Why It's Included
1	Engine Coolant Temperature	°C	The most direct indicator of overheating
2	Intake Manifold Absolute Pressure	kPa	Proxy for engine load — high load generates more heat
3	Engine RPM	RPM	High RPM sustained over time contributes to heat buildup
4	Intake Air Temperature	°C	Elevated intake air suggests heat soak under the hood
5	Air Flow Rate (MAF Sensor)	g/s	Indicates how hard the engine is working
6	Absolute Throttle Position	%	Driver demand — helps distinguish load-driven vs. cooling failure heating
7	Ambient Air Temperature	°C	Environmental baseline — hot days naturally raise all temperatures

Table 8.1: OBD-II features used by the LSTM Autoencoder

is low *and* throttle is minimal — which together suggest a cooling system failure rather than normal heat from heavy driving.

8.4 Implementation Details

8.4.1 Libraries and Frameworks

Library	Version	Purpose
TensorFlow / Keras	2.x	Model architecture definition, training, and inference
Scikit-learn	Latest	<code>MinMaxScaler</code> for feature normalization, <code>train_test_split</code> for data partitioning, <code>classification_report</code> for evaluation metrics
Pandas	Latest	Data loading, cleaning, pivoting, and preprocessing
NumPy	Latest	Numerical operations, sequence generation, threshold computation
Matplotlib / Seaborn	Latest	Visualization of error distributions, confusion matrices, and risk plots
Joblib	Latest	Serialization of the fitted scaler for deployment

Table 8.2: Libraries and frameworks used

8.4.2 Training Dataset

The model was trained on the **RADAR OBD-II Dataset** (sourced from Kaggle), which contains real-world driving telemetry collected from vehicles in Germany under three traffic conditions:

- **Normal traffic:** Standard urban and suburban driving
- **Free-flow traffic (Frei):** Highway/open road conditions with minimal congestion
- **Congested traffic (Stau):** Stop-and-go / heavy traffic conditions

This dataset was ideal for our purposes because it contains exclusively normal, healthy driving data — no mechanical faults were present during collection. This is exactly what an unsupervised anomaly detector needs: a clean baseline of “normal” to learn from.

8.4.3 Data Preprocessing Pipeline

The preprocessing is a multi-step pipeline designed to handle the messiness of real OBD-II data:

1. **Column name normalization:** The RADAR dataset had encoding issues (e.g., °C appearing as $\hat{\text{A}}^{\circ}\text{C}$ or $\tilde{\text{A}}, \hat{\text{A}}^{\circ}\text{C}$). A coalescing function identifies and merges these duplicate columns.
2. **Numeric conversion:** All feature columns are forced to numeric types. Non-parseable values are converted to NaN.
3. **Zero-spike handling:** Zero values in sensor readings (which typically indicate a momentary communication dropout, not an actual zero reading) are replaced with NaN.
4. **Interpolation:** Linear interpolation fills the NaN gaps, followed by forward-fill and backward-fill as fallbacks for edge cases. This produces a smooth, continuous signal.
5. **Feature scaling:** A `MinMaxScaler` is fitted globally across all training trips (using `partial_fit` in a first pass, then `transform` in a second pass). This normalizes all 7 features to the $[0, 1]$ range, ensuring no single feature dominates the reconstruction loss.
6. **Sliding window generation:** The continuous time series is sliced into overlapping windows of 50 timesteps each, with a hop of 5 timesteps. Each window becomes one training sample for the autoencoder.

8.4.4 Data Split

The data was split **at the trip level** (not at the individual window level) to prevent data leakage:

Splitting by trips ensures that windows from the same driving session never appear in both training and validation/test sets — which would artificially inflate performance metrics.

Split	Purpose	Proportion
Training	Learn normal reconstruction patterns	70% of trips
Validation	Monitor overfitting, compute anomaly thresholds	15% of trips
Test	Final evaluation with synthetic anomaly injection	15% of trips

Table 8.3: Data split strategy

8.4.5 Training Configuration

Parameter	Value
Epochs	80 (max)
Batch size	64
Optimizer	Adam (default learning rate)
Loss function	Mean Squared Error (MSE)
Early stopping	Patience = 8 epochs on validation loss
Model checkpoint	Saves best model by validation loss
Hardware	Kaggle cloud environment (GPU-accelerated)

Table 8.4: Training configuration

The model trains by reconstructing its own input — `model.fit(X_train, X_train)` — which is the defining characteristic of an autoencoder. Early stopping prevented unnecessary training: when validation loss stopped improving for 8 consecutive epochs, training halted and the best-performing weights were restored.

8.4.6 Anomaly Threshold Derivation

After training, anomaly detection thresholds were derived from the reconstruction errors on the **validation set** (which contains only normal driving data):

1. The trained model reconstructed every validation window
2. The MSE between input and reconstruction was computed for each window
3. Percentile-based thresholds were extracted from this error distribution:

Percentile	Threshold Value
95th (p95)	0.000304
99th (p99)	0.001103

Table 8.5: Anomaly thresholds derived from validation data

The intuition: if a new data window produces a reconstruction error higher than what 95% of known-normal data produces, it's likely not normal. The p95 threshold is more sensitive (catches more anomalies, but also more false positives), while p99 is stricter.

8.4.7 Scoring and Risk Classification

The raw reconstruction error is converted to an interpretable anomaly score between 0.0 and 1.0 using the formula:

$$score = \frac{error - p_{95}}{p_{99} \times relaxation_factor - p_{95}} \quad (8.1)$$

A relaxation factor of 2.5 is applied to the p99 boundary to account for domain shift when deploying on new vehicles. The score maps to risk levels:

Score Range	Risk Level	Interpretation
0.0 – 0.49	LOW	Normal operating conditions
0.5 – 0.79	MEDIUM	Unusual patterns detected — monitor closely
0.8 – 1.0	HIGH	Significant deviation from normal — investigate

Table 8.6: Risk classification mapping

8.4.8 Supplementary: Rule-Based Explanations

Beyond the ML model, we also implemented a **deterministic rules engine** (`rules_engine.py`) that runs alongside the LSTM. While the autoencoder provides a numerical anomaly score, the rules engine generates human-readable diagnostic explanations — things like “Coolant temperature is high while RPM is low — this pattern suggests a radiator fan or water pump malfunction.”

The rules engine checks for 8 specific conditions including:

- Absolute coolant temperature thresholds ($>100^{\circ}\text{C}$, $>105^{\circ}\text{C}$, $>110^{\circ}\text{C}$)
- Rate of coolant temperature rise
- Coolant-to-ambient temperature differential
- Overheating at idle (high coolant + low RPM)
- Intake air temperature anomalies
- MAP/MAF sensor sanity checks

This hybrid approach — statistical ML + domain-specific rules — gives us both a quantitative risk score and qualitative, actionable diagnostic information that a driver or technician can actually act on.

8.5 Performance Metrics

8.5.1 Evaluation Approach

Evaluating an unsupervised anomaly detector is fundamentally different from evaluating a supervised classifier, because the model never sees labeled anomalies during training. To properly evaluate performance, a **synthetic test set** was created:

1. Normal test windows were taken directly from the held-out test trips (15% of data)
2. Overheating events were **synthetically injected** into 15% of these normal windows by applying a gradual coolant temperature ramp (a 25% increase in normalized scale over 60% of the window length)
3. An equal number of unmodified normal windows were randomly sampled for balance
4. The combined set was shuffled for evaluation

This gives us a labeled test set where we know the ground truth — allowing standard classification metrics to be computed.

8.5.2 Metrics on the Synthetic Test Set

Using the p95 threshold from the validation set, the model’s predictions on the synthetic test set were evaluated using scikit-learn’s `classification_report`. The model successfully distinguished between normal windows and windows with injected overheating ramps — the reconstruction error for overheating windows was measurably higher than for normal windows, cleanly separating the two classes around the p95 threshold.

The error distribution histogram confirmed this separation visually: normal windows clustered tightly at low reconstruction errors, while the overheating-injected windows produced a distinct, higher-error cluster with clear separation from the normal distribution.

8.5.3 Real-World Deployment Results

The trained model was then deployed against **real telemetry data** collected from our Raspberry Pi OBD-II setup — 15,487 readings across 7 driving sessions, producing 3,013 analysis windows:

Metric	Value
Total driving sessions analyzed	7
Total analysis windows processed	3,013
Mean reconstruction error (across all windows)	0.026114
Maximum reconstruction error	0.042249
p95 threshold (from training)	0.000304
p99 threshold (from training)	0.001103

Table 8.7: Real-world deployment metrics

Risk classification breakdown:

Risk Level	Sessions	Percentage
LOW	0	0%
MEDIUM	0	0%
HIGH	7	100%

Table 8.8: Risk classification results on real-world data

8.5.4 Interpreting the Results

At first glance, the fact that all 7 sessions were classified as HIGH risk might seem alarming — or like the model is broken. But this result is actually **expected and explainable**, and understanding why is arguably more valuable than a perfect accuracy number.

The core issue is domain shift. The LSTM Autoencoder was trained on the RADAR OBD-II dataset — data from German vehicles, driving on German roads, in European climate conditions. Our Raspberry Pi collected data from a different vehicle entirely, driven in Pakistani traffic conditions, with different ambient temperatures, driving patterns, and engine characteristics.

The model learned that “normal” looks like a very specific set of patterns from the training data. When it encounters data from a completely different vehicle and environment — even if that vehicle is operating perfectly normally — it can’t reconstruct those unfamiliar patterns well. The reconstruction error (0.026–0.042) is roughly 30–40× higher than the thresholds (0.000304–0.001103), which is consistent with the model encountering a fundamentally different data distribution, not necessarily with the vehicle actually overheating.

This is a well-known challenge in machine learning called **distribution shift** or **covariate shift**, and it’s particularly pronounced for autoencoders because they are highly sensitive to the specific statistical characteristics of their training data.

8.5.5 Challenges and Limitations

1. **Feature gap:** Our Pi OBD2 system natively collects 5 of the 7 features the model expects. The missing two — Intake Air Temperature and Mass Air Flow Rate — were synthesized with realistic estimates. While these estimates are reasonable, they introduce a source of noise that contributes to higher reconstruction errors.
2. **Computational constraints:** Running LSTM inference on the Raspberry Pi 3’s ARM Cortex-A53 CPU is significantly slower than the Kaggle GPU used for training. Processing 3,013 windows took approximately 30 minutes. For real-time deployment, the model would need optimization — conversion to TensorFlow Lite, quantization, or batch inference.
3. **Threshold calibration:** The most impactful improvement would be to collect 2–3 hours of verified normal driving data from the target vehicle and either (a) fine-tune the model on this data, or (b) recompute the p95/p99 thresholds from vehicle-specific validation data. This would effectively eliminate the domain shift problem and produce meaningful, calibrated risk scores.

8.5.6 What This Means Practically

Despite the domain shift producing uniformly HIGH classifications on our specific vehicle, the underlying model is sound:

- It **correctly separates** normal from overheating data in its training domain (synthetic test set evaluation)

- It **correctly identifies** that our Pi data is statistically different from its training distribution (which is technically true)
- The **infrastructure is in place** — the inference pipeline, scoring system, and rules engine all work end-to-end

The path to production-ready anomaly detection is clear: collect a baseline of normal driving data from the target vehicle, retrain or fine-tune the thresholds, and the system becomes a calibrated, vehicle-specific overheating predictor.